BCSE307P – Compiler Design Lab

24BCI0130
Bhoomi Annamdevara

Lab Assignment – 4

Title: Construct a Predictive Parsing Table for the given CFG

Aim:

To write a C program that reads a context-free grammar, computes the FIRST and FOLLOW sets of every non-terminal, and constructs the Predictive (LL(1)) Parsing Table M[A, a]. The program lists every table entry, prints the table with non-terminals as rows and terminals (with $) as columns, and reports whether the grammar is LL(1) by checking for cells with more than one production.

Algorithm:

1. Start.
2. Read the number of productions and each production in the form A->α ('#' denotes ε). The LHS of the first production is taken as the start symbol.
3. Read the list of non-terminals and terminals; append $ as the end-marker column.
4. Compute FIRST of every non-terminal by repeating until no set changes: for each production A->Y1Y2…Yk, add FIRST(Y1)−{ε} to FIRST(A); if Y1 can derive ε continue with Y2, and so on. If every Yi can derive ε (or α = ε), add ε to FIRST(A). For a terminal a, FIRST(a) = {a}.
5. Compute FOLLOW by repeating until no set changes: put $ in FOLLOW(start). For each production A->αBβ, add FIRST(β)−{ε} to FOLLOW(B); if β is empty or derives ε, add FOLLOW(A) to FOLLOW(B).
6. For each production A->α: for every terminal a in FIRST(α) add A->α to M[A, a]; if ε ∈ FIRST(α), add A->α to M[A, b] for every b ∈ FOLLOW(A) (including $).
7. Print FIRST sets, FOLLOW sets, all table entries and the parsing table. If any cell holds more than one production, report "NOT LL(1)", otherwise "LL(1)".
8. Stop.

Program:

```c
/*
 * BCSE307P - Compiler Design Lab | Lab Assignment 4 (a)
 * Construction of Predictive (LL(1)) Parsing Table
 * Name   : Bhoomi Annamdevara
 * Reg No : 24BCI0130
 */
#include <stdio.h>
#include <string.h>

#define MAXP 30
#define MAXS 20
#define MAXN 20
#define EPS '#'

char lhs[MAXP], rhs[MAXP][MAXS];
int np, nn, nt;
char NT[MAXN], T[MAXN + 1];            /* T[nt] = '$' */
int first[MAXN][MAXN + 1], firstEps[MAXN];
int follow[MAXN][MAXN + 1];
int table[MAXN][MAXN + 1][MAXP], cnt[MAXN][MAXN + 1];

int ntIdx(char c) { for (int i = 0; i < nn; i++) if (NT[i] == c) return i; return -1; }
int tIdx(char c)  { for (int i = 0; i <= nt; i++) if (T[i] == c) return i; return -1; }

/* FIRST of rhs[p] starting at position k; returns 1 if it can derive epsilon */
int firstOfString(int p, int k, int out[]) {
    for (; rhs[p][k]; k++) {
        char s = rhs[p][k];
        if (s == EPS) continue;
        int A = ntIdx(s);
        if (A < 0) { int t = tIdx(s); if (t >= 0) out[t] = 1; return 0; }
        for (int j = 0; j < nt; j++) if (first[A][j]) out[j] = 1;
        if (!firstEps[A]) return 0;
    }
    return 1;
}

void computeFirst(void) {
    int changed = 1;
    while (changed) {
        changed = 0;
        for (int p = 0; p < np; p++) {
            int A = ntIdx(lhs[p]), tmp[MAXN + 1] = {0};
            int e = firstOfString(p, 0, tmp);
            for (int j = 0; j < nt; j++)
                if (tmp[j] && !first[A][j]) first[A][j] = changed = 1;
            if (e && !firstEps[A]) firstEps[A] = changed = 1;
        }
    }
}

void computeFollow(void) {
    follow[ntIdx(lhs[0])][nt] = 1;      /* $ in FOLLOW(start) */
    int changed = 1;
    while (changed) {
        changed = 0;
        for (int p = 0; p < np; p++) {
            int A = ntIdx(lhs[p]);
            for (int k = 0; rhs[p][k]; k++) {
                int B = ntIdx(rhs[p][k]);
```

```c
                if (B < 0) continue;
                int tmp[MAXN + 1] = {0};
                int e = firstOfString(p, k + 1, tmp);
                for (int j = 0; j < nt; j++)
                    if (tmp[j] && !follow[B][j]) follow[B][j] = changed = 1;
                if (e)
                    for (int j = 0; j <= nt; j++)
                        if (follow[A][j] && !follow[B][j]) follow[B][j] = changed = 1;
            }
        }
    }
}

void addEntry(int A, int t, int p) {
    for (int i = 0; i < cnt[A][t]; i++) if (table[A][t][i] == p) return;
    table[A][t][cnt[A][t]++] = p;
}

void buildTable(void) {
    for (int p = 0; p < np; p++) {
        int A = ntIdx(lhs[p]), tmp[MAXN + 1] = {0};
        int e = firstOfString(p, 0, tmp);
        for (int j = 0; j < nt; j++) if (tmp[j]) addEntry(A, j, p);
        if (e) for (int j = 0; j <= nt; j++) if (follow[A][j]) addEntry(A, j, p);
    }
}

void printSet(const char *name, char X, int set[], int eps, int withDollar) {
    int f = 1;
    printf("%s(%c) = { ", name, X);
    for (int j = 0; j < (withDollar ? nt + 1 : nt); j++)
        if (set[j]) { printf(f ? "%c" : ", %c", T[j]); f = 0; }
    if (eps) printf(f ? "#" : ", #");
    printf(" }\n");
}

int main(void) {
    printf("=====================================================\n");
    printf("  Predictive Parsing Table Construction - LL(1)\n");
    printf("  Name : Bhoomi Annamdevara   Reg No : 24BCI0130\n");
    printf("=====================================================\n\n");

    printf("No.of productions : ");
    scanf("%d", &np);
    for (int i = 0; i < np; i++) {
        char buf[64];
        printf("Enter productions%d: ", i + 1);
        scanf("%s", buf);                       /* format: A->alpha */
        lhs[i] = buf[0];
        strcpy(rhs[i], buf + 3);
    }
    printf("No.of non terminals   : ");
    scanf("%d", &nn);
    for (int i = 0; i < nn; i++) { printf("Enter Non Terminal%d   : ", i + 1); scanf(" %c", &NT[i]); }
    printf("No.of terminals   : ");
    scanf("%d", &nt);
    for (int i = 0; i < nt; i++) { printf("Enter Terminals%d   : ", i + 1); scanf(" %c", &T[i]); }
    T[nt] = '$';

    computeFirst();
```

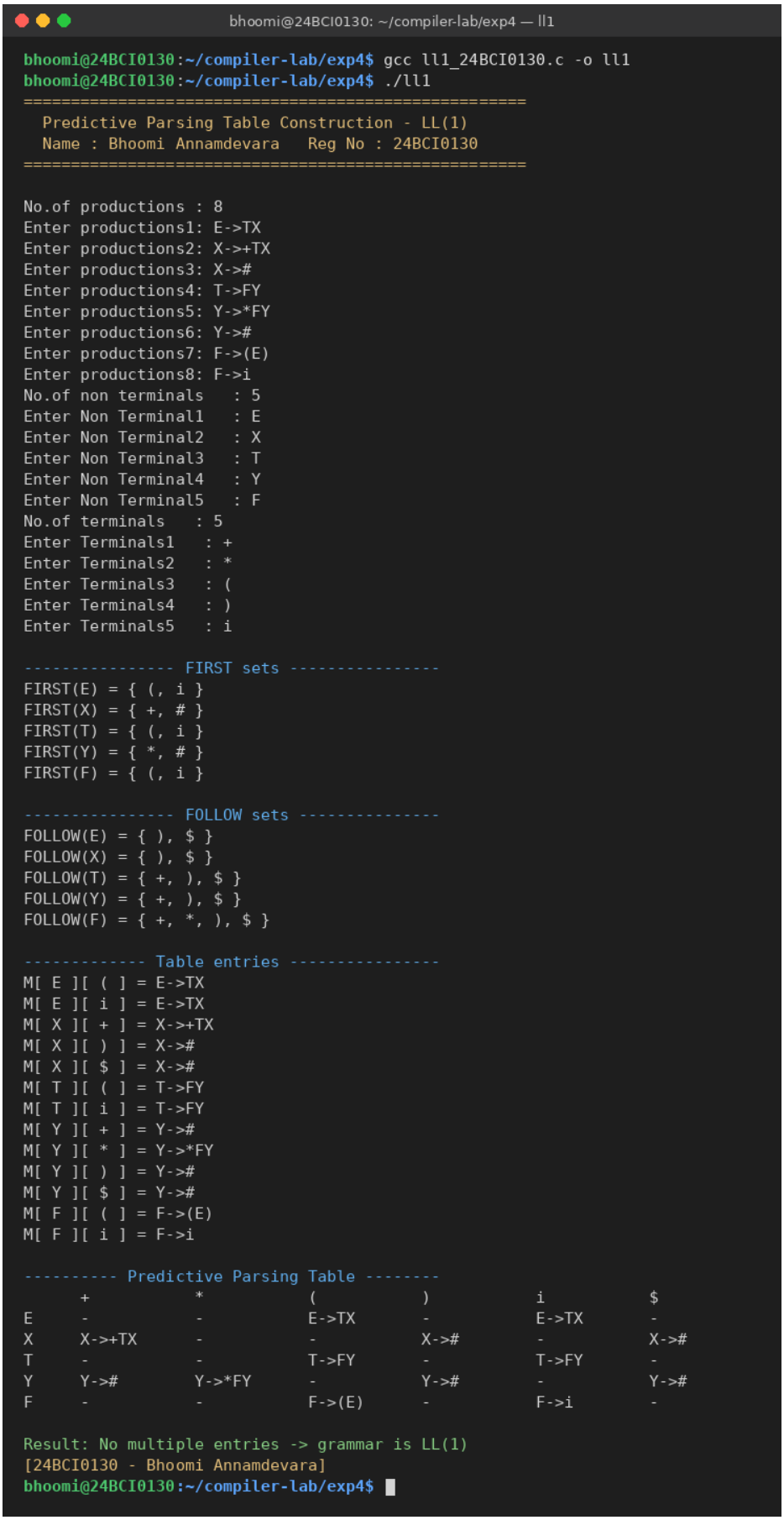
```
    computeFollow();
    buildTable();

    printf("\n---------------- FIRST sets ----------------\n");
    for (int i = 0; i < nn; i++) printSet("FIRST", NT[i], first[i], firstEps[i], 0);
    printf("\n---------------- FOLLOW sets ----------------\n");
    for (int i = 0; i < nn; i++) printSet("FOLLOW", NT[i], follow[i], 0, 1);

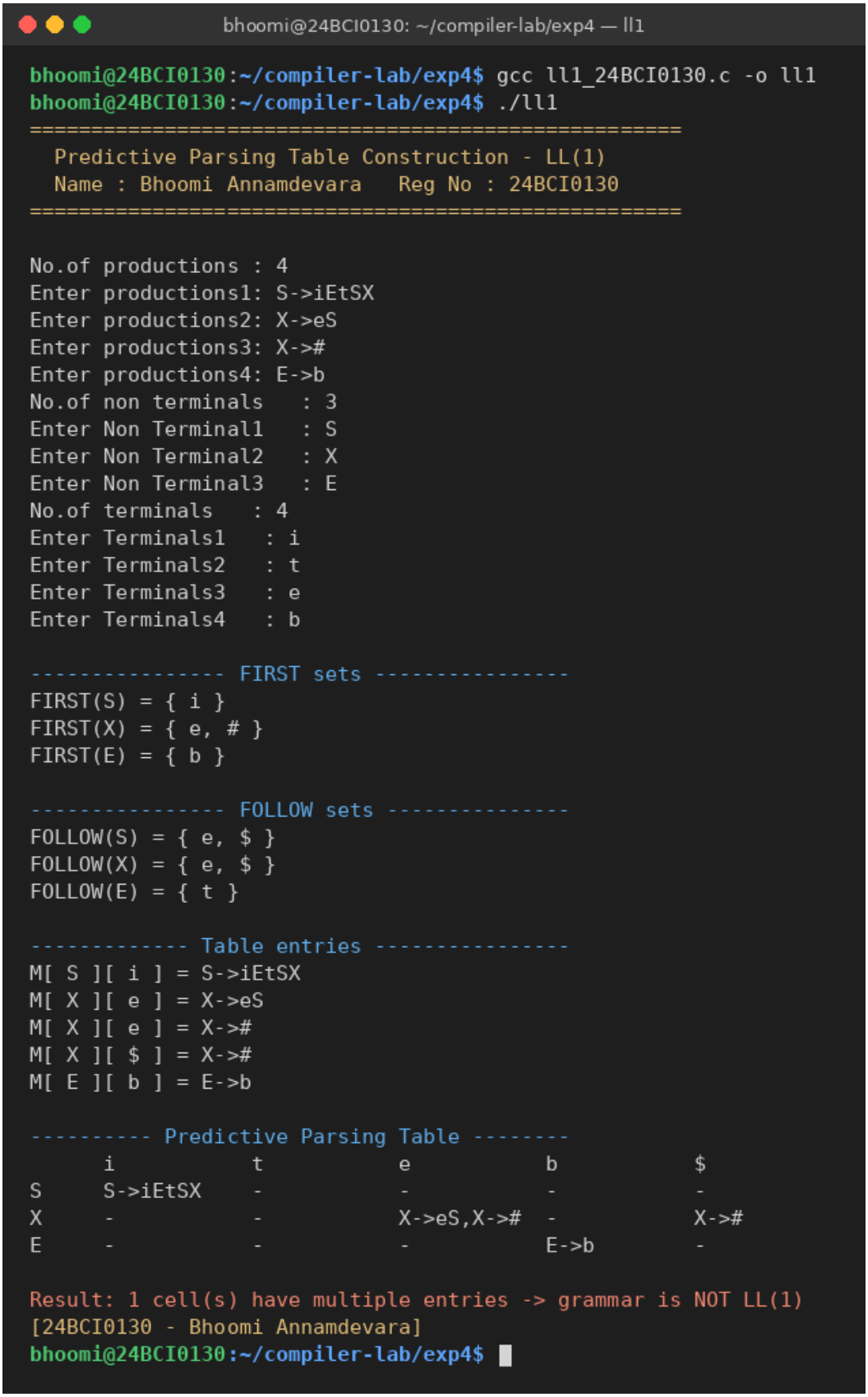
    printf("\n------------- Table entries ----------------\n");
    int conflicts = 0;
    for (int i = 0; i < nn; i++)
        for (int j = 0; j <= nt; j++) {
            for (int k = 0; k < cnt[i][j]; k++) {
                int p = table[i][j][k];
                printf("M[ %c ][ %c ] = %c->%s\n", NT[i], T[j], lhs[p], rhs[p]);
            }
            if (cnt[i][j] > 1) conflicts++;
        }

    printf("\n---------- Predictive Parsing Table --------\n%-6s", "");
    for (int j = 0; j <= nt; j++) printf("%-12c", T[j]);
    printf("\n");
    for (int i = 0; i < nn; i++) {
        printf("%-6c", NT[i]);
        for (int j = 0; j <= nt; j++) {
            char cell[64] = "-";
            if (cnt[i][j]) {
                cell[0] = 0;
                for (int k = 0; k < cnt[i][j]; k++) {
                    int p = table[i][j][k];
                    char one[24];
                    sprintf(one, "%s%c->%s", k ? "," : "", lhs[p], rhs[p]);
                    strcat(cell, one);
                }
            }
            printf("%-12s", cell);
        }
        printf("\n");
    }
    if (conflicts)
        printf("\nResult: %d cell(s) have multiple entries -> grammar is NOT LL(1)\n", conflicts);
    else
        printf("\nResult: No multiple entries -> grammar is LL(1)\n");
    printf("[24BCI0130 - Bhoomi Annamdevara]\n");
    return 0;
}
```

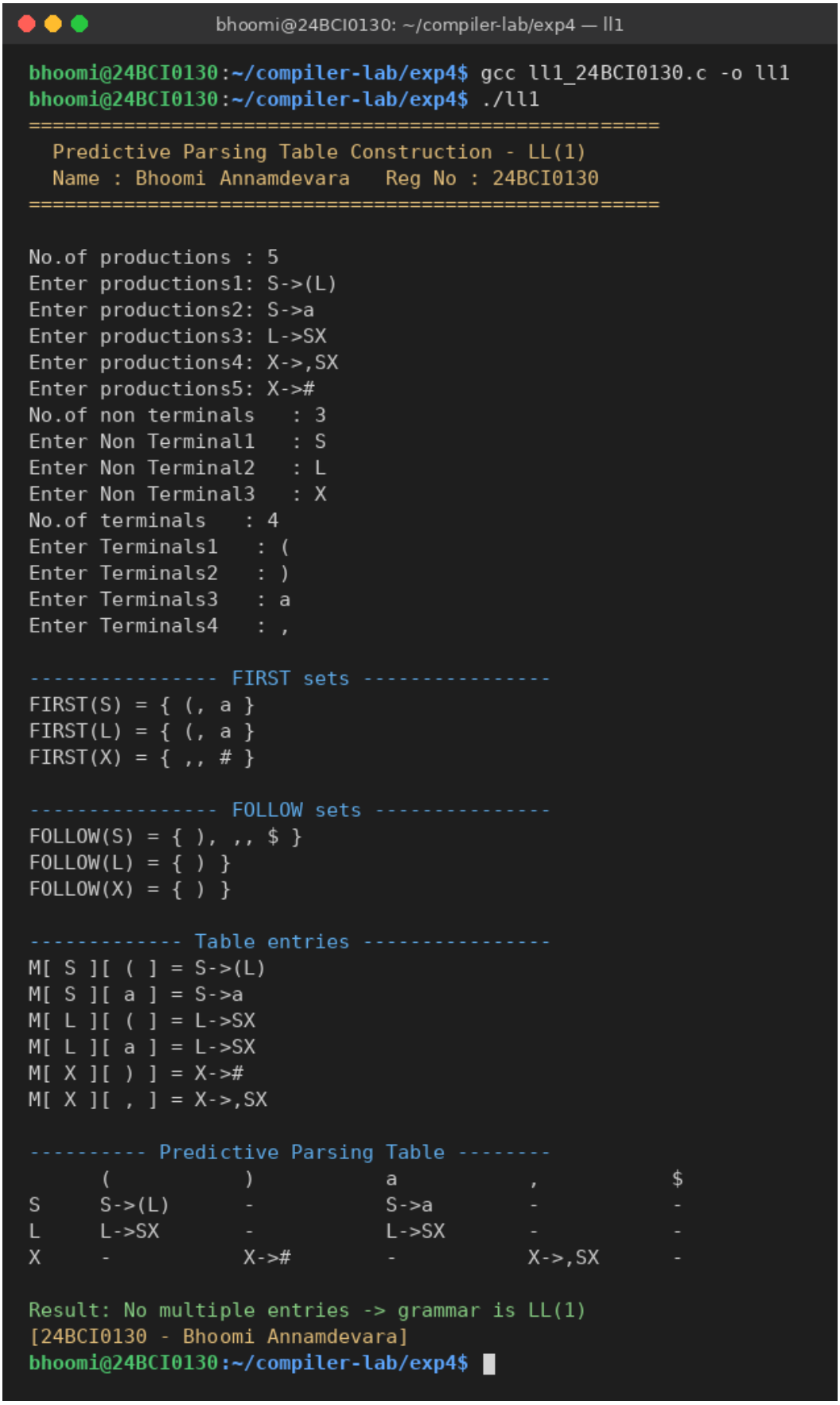
Input / Output Screenshots:



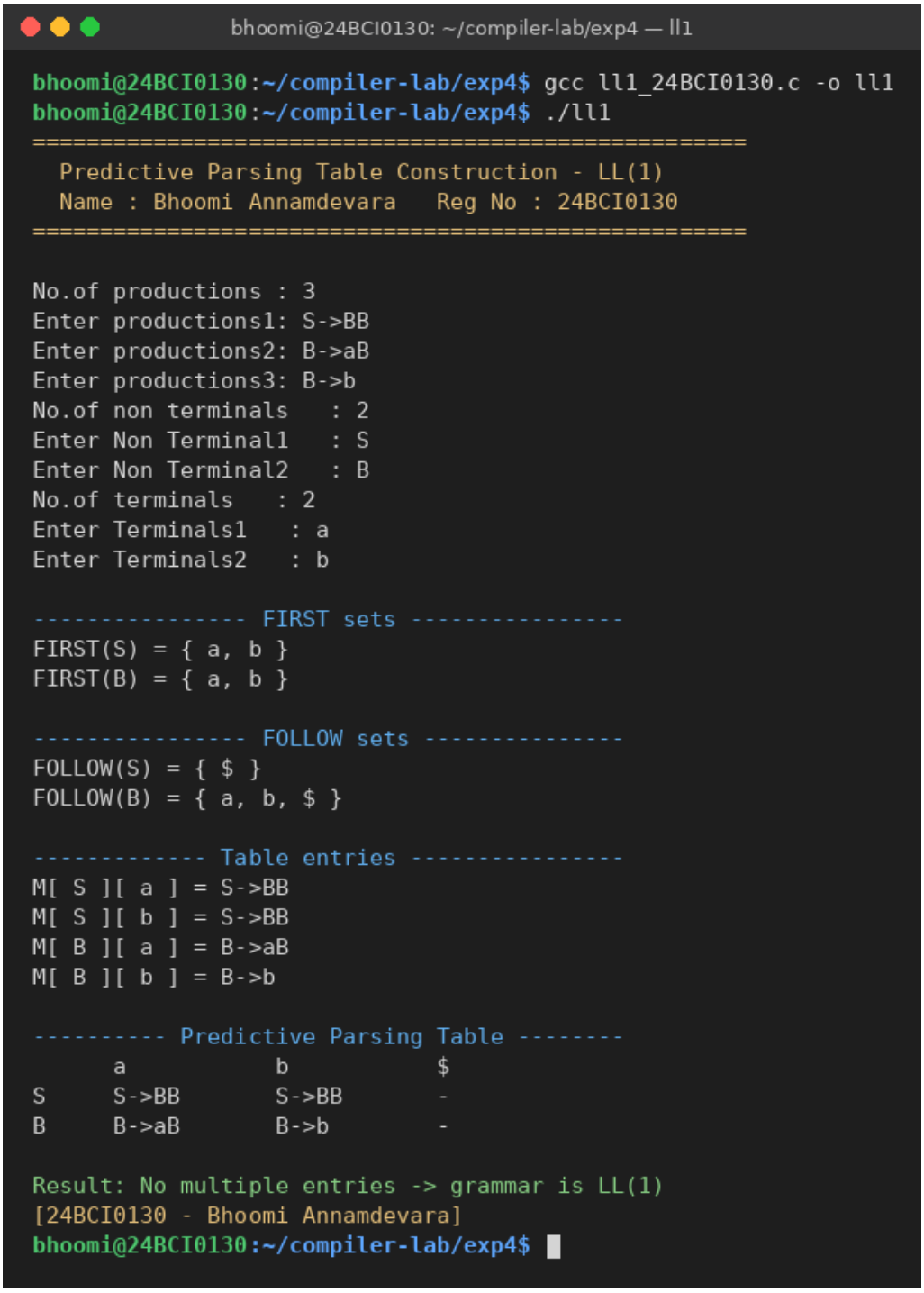
Run 1 – Expression grammar E->TX, X->+TX | #, T->FY, Y->*FY | #, F->(E) | i (LL(1))



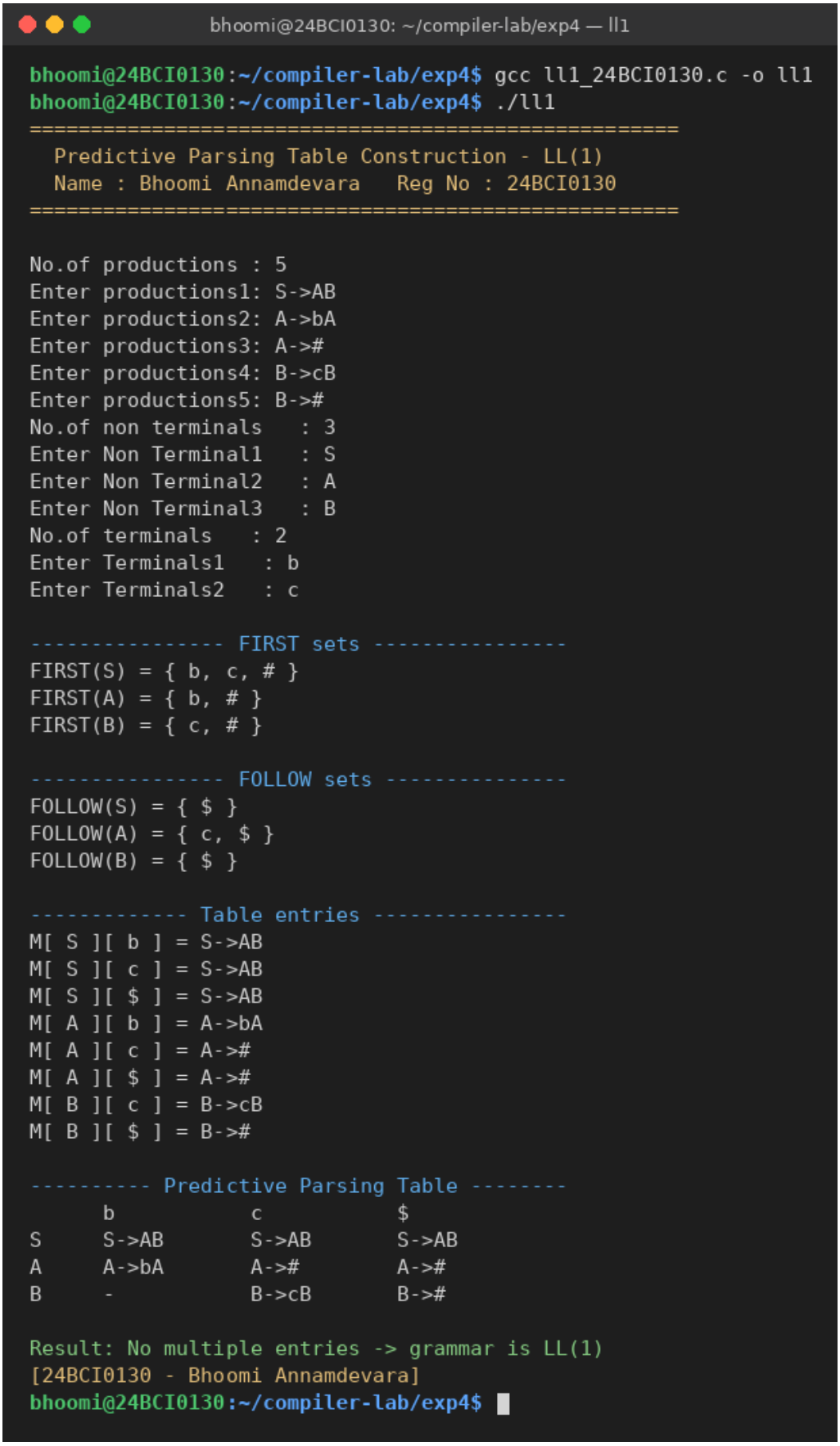
Run 2 – Dangling-else grammar S->iEtSX, X->eS | #, E->b (M[X][e] has two entries → NOT LL(1))



Run 3 – List grammar S->(L) | a, L->SX, X->,SX | # (LL(1))



Run 4 – Grammar S->BB, B->aB | b (LL(1))



Run 5 – Grammar S->AB, A->bA | #, B->cB | # (LL(1))

Result:

The C program was compiled with gcc and executed successfully for five grammars. The FIRST and FOLLOW sets produced by the program agree with those computed by hand, and the predictive parsing table was constructed correctly. Grammars 1, 3, 4 and 5 produced at most one production per cell and are LL(1); for the dangling-else grammar (Run 2) the cell M[X, e] contains both X->eS and X->#, so the program correctly reports that the grammar is not LL(1).

Title: Type Checking for the given Expression

Aim:

To write a C program that reads a single input string containing variable declarations followed by assignment expressions, builds a symbol table from the declarations, and checks whether the type of every operand on the right-hand side of each assignment matches the type of the left-hand side variable, printing MATCHED or NOT MATCHED.

Algorithm:

1 Start.
2 Read the input string, e.g. int a,b,c;float d;char e,f;a=b+c;d=e+d;
3 Split the string into statements at every ';'.
4 For each statement, read the first word. If it is a data type (int / float / char / double), split the rest at ',' and insert each name with that type into the symbol table.
5 Otherwise the statement is an assignment lhs = expr. Look up the type of lhs; if lhs is not declared, print an error.
6 Split expr into operands at the operators + - * / % ( ). For each operand find its type: from the symbol table if it is an identifier, else from its lexical form (digits → int, digits with one '.' → float, quoted → char).
7 Compare the operand type with the lhs type and print "type of lhs & operand -->MATCHED" or "-->NOT MATCHED".
8 Repeat for all statements, then stop.

Program:

```c
/*
 * BCSE307P - Compiler Design Lab | Lab Assignment 4 (b)
 * Type Checking for the given Expression
 * Name   : Bhoomi Annamdevara
 * Reg No : 24BCI0130
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXV 50

char name[MAXV][32], type[MAXV][8];
int nv = 0;

const char *typeOf(const char *id) {
    for (int i = 0; i < nv; i++) if (strcmp(name[i], id) == 0) return type[i];
    /* constants: infer type from lexical form */
    int digits = 1, dot = 0;
    for (int i = 0; id[i]; i++) {
        if (id[i] == '.') dot++;
        else if (!isdigit((unsigned char)id[i])) digits = 0;
    }
    if (digits && dot == 0) return "int";
    if (digits && dot == 1) return "float";
    if (id[0] == '\'') return "char";
    return "undeclared";
}

int isType(const char *w) {
    return !strcmp(w, "int") || !strcmp(w, "float") || !strcmp(w, "char") || !strcmp(w, "double");
}

/* handle one statement (text between ';') */
void processStmt(char *s) {
    while (isspace((unsigned char)*s)) s++;
    if (!*s) return;

    char first[32]; int k = 0;
    while (isalnum((unsigned char)*s) || *s == '_') first[k++] = *s++;
    first[k] = 0;

    if (isType(first)) {                          /* declaration */
        char *tok = strtok(s, ", \t");
        while (tok) {
            strcpy(name[nv], tok); strcpy(type[nv], first); nv++;
            tok = strtok(NULL, ", \t");
        }
        return;
    }
    /* assignment: first = operand op operand ... */
    const char *lt = typeOf(first);
    if (!strcmp(lt, "undeclared")) { printf("ERROR: %s is not declared\n", first); return; }
    char *eq = strchr(s, '=');
    if (!eq) return;
    char *tok = strtok(eq + 1, "+-*/%() \t");
    while (tok) {
        const char *rt = typeOf(tok);
        printf("type of %s & %s -->%s\n", first, tok,
               strcmp(lt, rt) == 0 ? "MATCHED" : "NOT MATCHED");
```

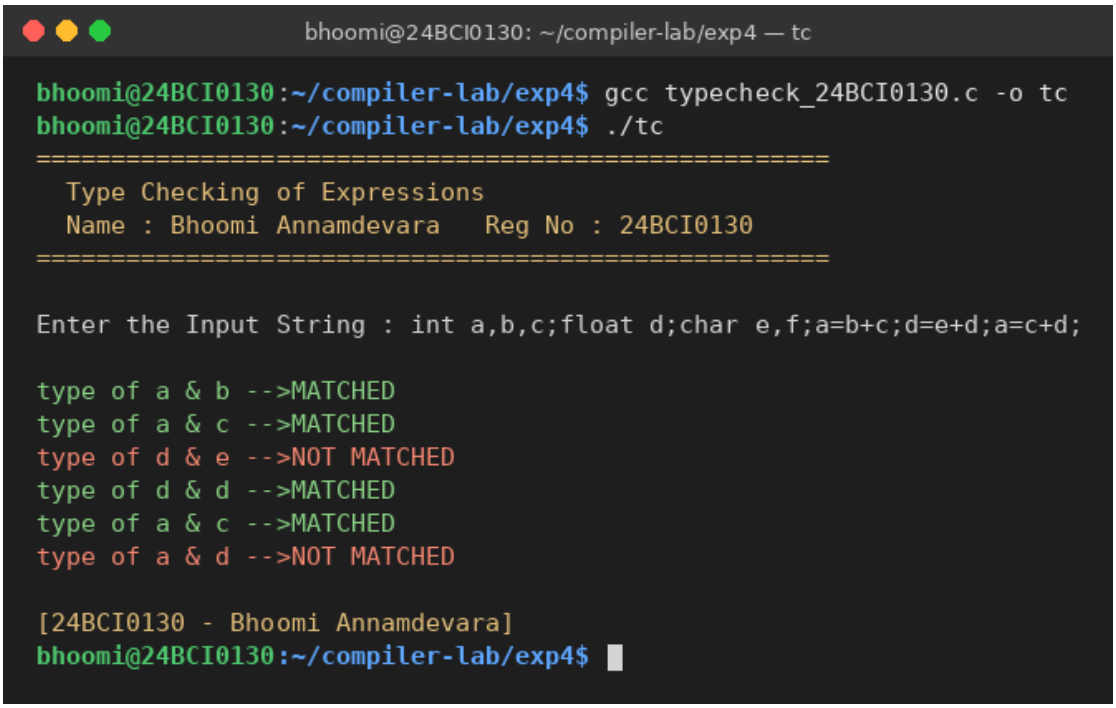
```c
        tok = strtok(NULL, "+-*/%() \t");
    }
}

int main(void) {
    char input[1024], *stmts[100];
    int ns = 0;
    printf("=========================================================\n");
    printf("  Type Checking of Expressions\n");
    printf("  Name : Bhoomi Annamdevara   Reg No : 24BCI0130\n");
    printf("=========================================================\n\n");
    printf("Enter the Input String : ");
    fgets(input, sizeof input, stdin);
    input[strcspn(input, "\n")] = 0;
    printf("\n");

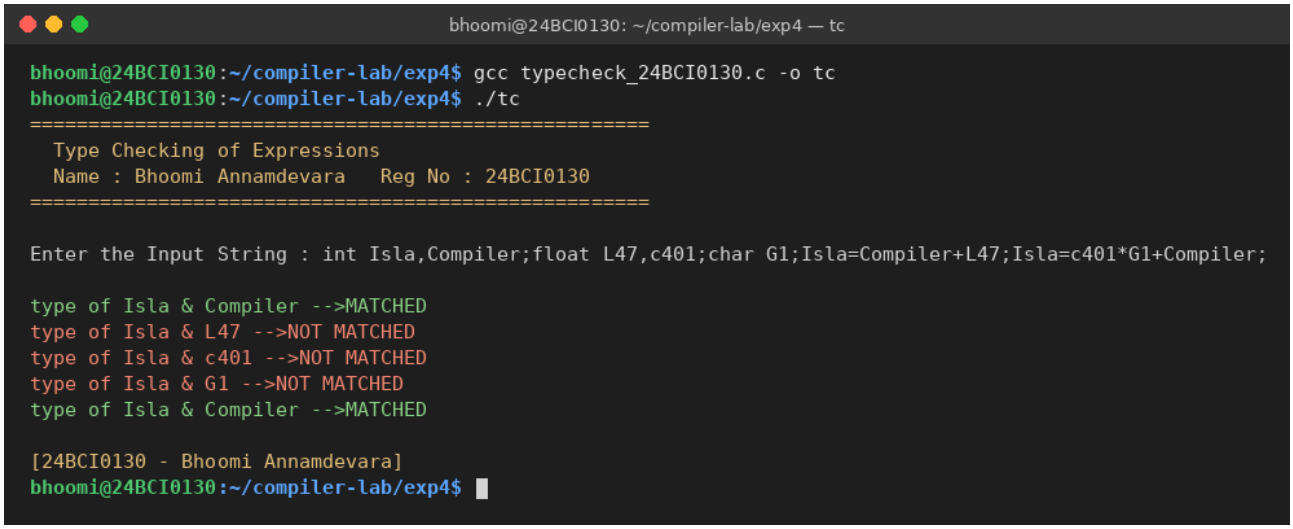
    /* split on ';' first, since processStmt uses strtok internally */
    for (char *p = strtok(input, ";"); p; p = strtok(NULL, ";")) stmts[ns++] = p;
    for (int i = 0; i < ns; i++) processStmt(stmts[i]);

    printf("\n[24BCI0130 - Bhoomi Annamdevara]\n");
    return 0;
}
```

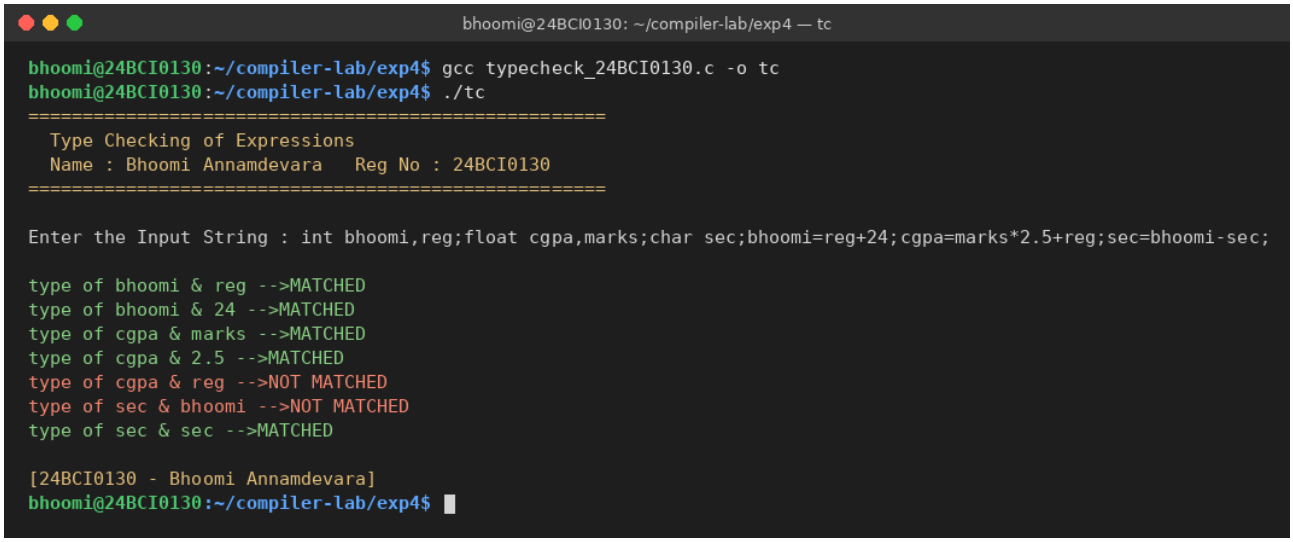
Input / Output Screenshots:



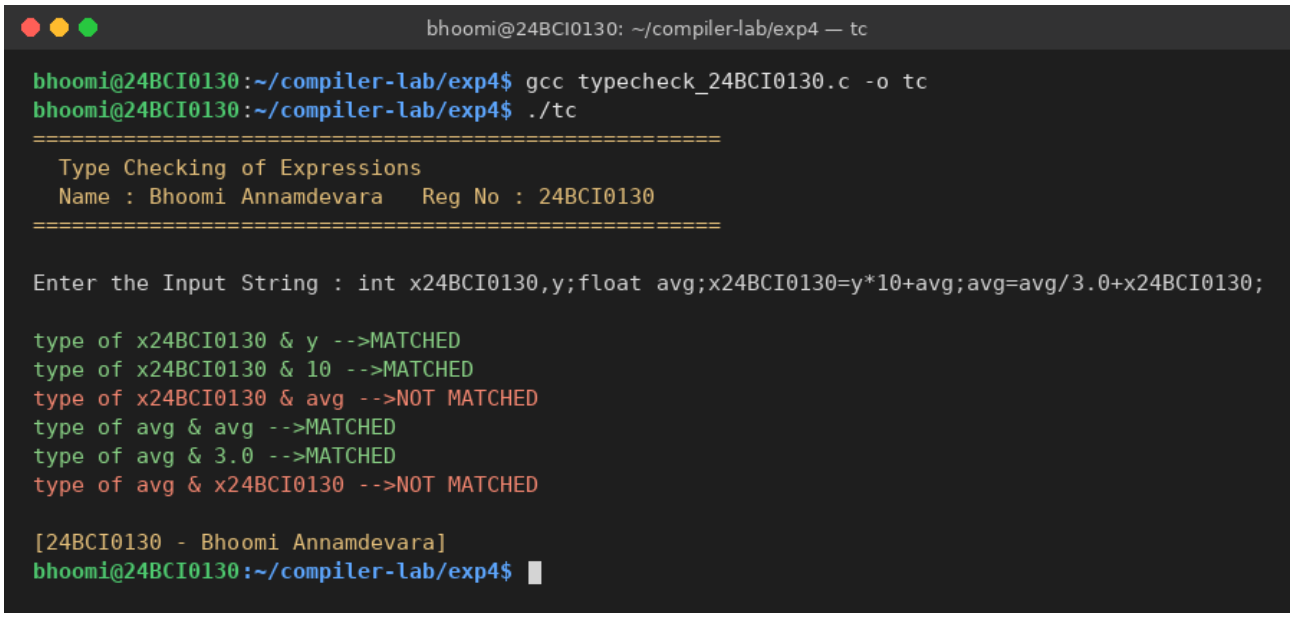
Run 1 – int a,b,c; float d; char e,f; with three assignments



Run 2 – identifiers Isla, Compiler, L47, c401, G1



Run 3 – identifiers bhoomi, reg, cgpa, marks, sec with int and float constants



Run 4 – identifier x24BCI0130 (register number) with mixed int/float operands

Result:

The C program was compiled with gcc and executed successfully for four input strings. Declared identifiers were stored in the symbol table with their types, the types of numeric constants were inferred from their lexical form, and each right-hand-side operand was correctly reported as MATCHED or NOT MATCHED against the type of the left-hand-side variable.